Kuramoto SNN Object Recognition

Anonymous submission

Abstract

1 Introduction

2 Method

Our model generates a separate mask for each discovered object by grouping image patches with similar temporal spike patterns. Inspired by S^2-Net (Dan and Wu 2026), it converts an image into patch activations and uses coupled oscillators to regulate when patches spike. Figure 1 shows the signal flow.

2.1 From Images to Neural Activations

Patch activation. We encode an RGB image with eight convolutional filters, normalize and bound the feature maps, and pool them into patch activations. The patch activation, γ_i, represents patch i. The patch indices, i and j, identify image patches; the component index, a, identifies an oscillator component; and the time index, t, identifies a simulation step.

Patch connection graph. To connect related patches, we combine feature agreement with the paths between them.

$$S_{ij} = \frac{\boldsymbol{z}_i^\top \boldsymbol{z}_j}{\tau} - r\, d(i,j). \tag{1}$$

The connection score, S_{ij}, compares the normalized patch embeddings, $\boldsymbol{z}_i$ and $\boldsymbol{z}_j$, using the temperature, τ, and penalizes the geodesic distance, $d(i,j)$, using the spatial cost weight, r. τ and r are derived from trainable scalars.

We normalize the strongest connections and make their weights reciprocal.

$$\begin{aligned} B_{ij} &= G\ \mathrm{softmax}_{j\in\mathcal{N}_i}(S_{ij}), \\ A_{ij} &= \frac{B_{ij} + B_{ji}}{2}. \end{aligned} \tag{2}$$

For each patch i, the neighbor set, $\mathcal{N}_i$, contains the 512 patches with the highest connection scores, S_{ij}. The connection gain, G, is derived from a trainable scalar. The normalized connection weight, B_{ij}, is zero for unselected neighbors. The symmetric connection weight, A_{ij}, defines the patch graph.

Input phase. A bounded phase input supplies image information to each oscillator.

$$\begin{aligned} \boldsymbol{u}_i &= W_{\mathrm{drive}}\gamma_i, \\ \boldsymbol{\phi}_i &= \pi \tanh(W_\phi\, \bar{\boldsymbol{u}}_i), \qquad \boldsymbol{\theta}_i(0) = \boldsymbol{\phi}_i. \end{aligned} \tag{3}$$

The projection matrix, $W_{\mathrm{drive}} \in \mathbb{R}^{4\times 8}$, and the phase gain, $W_\phi \in \mathbb{R}$, are trainable weights. The projected input, $\boldsymbol{u}_i$, is standardized to obtain $\bar{\boldsymbol{u}}_i$. The sensory drive, $\boldsymbol{\phi}_i$, initializes the oscillator phase, $\boldsymbol{\theta}_i(0)$.

2.2 Recurrent Core

Kuramoto layer. Oscillators coordinate their phases through intrinsic drift, patch interactions, and sensory drive.

$$\begin{aligned} \boldsymbol{\theta}_i(t+1) = \boldsymbol{\theta}_i(t) + \Delta t\Big(&\boldsymbol{\omega}_i + f\boldsymbol{\phi}_i \\ &+ \boldsymbol{\kappa}_i \odot \sin(\boldsymbol{\phi}_i - \boldsymbol{\theta}_i(t)) \\ &+ \frac{K}{N}\sum_j \bar{A}_{ij} \sin\big(\boldsymbol{\theta}_j(t) - \boldsymbol{\theta}_i(t) - \alpha_{ij}\big)\Big). \end{aligned} \tag{4}$$

The **natural frequency**, $\boldsymbol{\omega}_i$, gives each oscillator its intrinsic rate of phase change. The **coupling term**, the sum over j, adjusts the phase according to other patches, using the corrected connection weight, $\bar{A}_{ij}$, and the phase lag, α_{ij}. The **drive term** uses the sensory drive, $\boldsymbol{\phi}_i$, to adjust frequency through f and to pull the phase toward the input when the drive strength, $\boldsymbol{\kappa}_i$, is positive. The coupling scale, K, is normalized by the patch count, N. The time step, Δt, scales the update, and $\odot$ denotes elementwise multiplication. The parameters $\boldsymbol{\omega}_i$, f, and $\boldsymbol{\kappa}_i$ are trainable.

Sinusoidal gating. The patch rhythm controls how oscillatory signals enter the spiking core.

$$\begin{aligned} \bar{\theta}_i(t) &= \frac{1}{4}\sum_{a=1}^{4} \theta_{ia}(t), \\ g_i(t) &= \frac{1 + \sin(\bar{\theta}_i(\max(0, t-2)))}{2}. \end{aligned} \tag{5}$$

$$\boldsymbol{x}_i(t) = g_i(t)\sin(\boldsymbol{\theta}_i(t)). \tag{6}$$

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

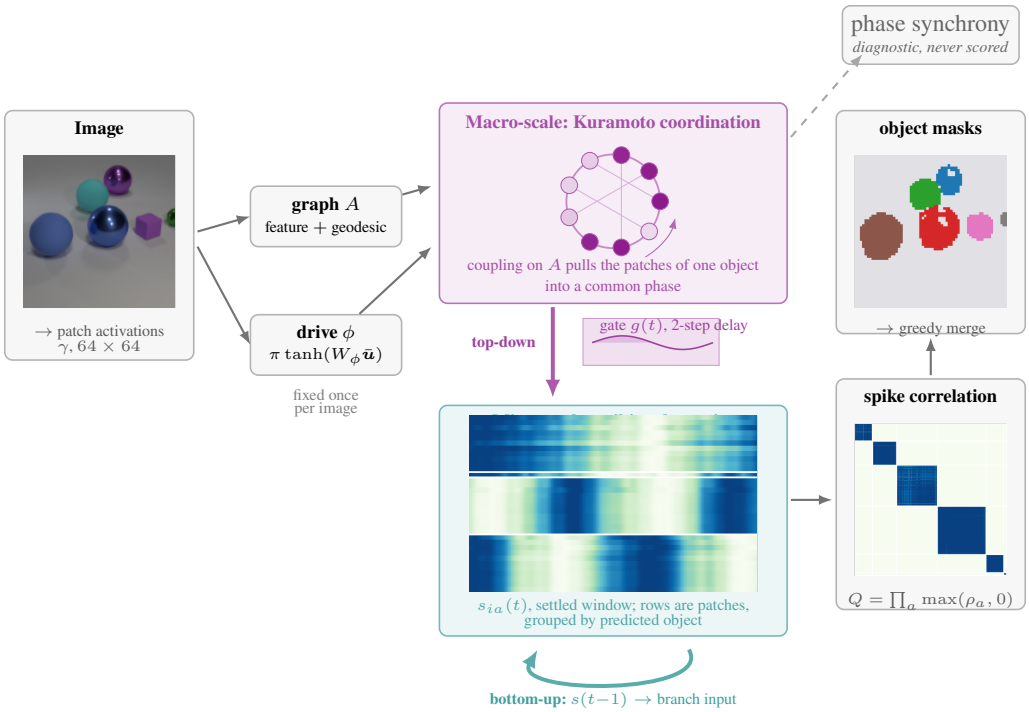


Figure 1: Signal flow, with the image, spiking, correlation and mask panels showing the trained model's actual output on one CLEVR image from the reserved split (`image_id` 1355, six objects; FG-ARI 0.9158, the median of the 121 reserved scenes with four to six objects, against a split mean of 0.8421). The mask shown is bit-identical to the one that was scored. An image is encoded into patch activations γ, which fix both the coupling graph A and the sensory drive ϕ once per image. Oscillator phases θ evolve under Kuramoto coupling on A (ring, illustrative). The mean patch phase gates the spiking layer two steps later (**top-down**, violet), and within that layer the previous spike re-enters the dendritic branch input (**bottom-up**, teal). In the spiking panel, patches the readout assigns to one object share a band pattern and different objects are offset; the correlation panel is that structure as the affinity Q, ordered by predicted object, and its blocks are the groups that become masks. The mask panel has the minimum-region-size rule applied ($f = 0.01$, fixed on validation): without it this image yields eleven regions of sizes $285, 240, 141, 106, 102, 16, 1, 1, 1, 1, 1$, and the rule removes the five single-patch ones, leaving six for six true objects.

The mean patch phase, $\bar{\theta}_i(t)$, summarizes the four oscillator phases in a patch. The delayed gate, $g_i(t)$, controls signal transmission using the patch phase from two steps earlier. The gated phase feature, $\boldsymbol{x}_i(t)$, carries the modulated oscillations to the dendritic layer.

Dendritic layer. Four branches process the rhythmic input and previous spike before combining their responses.

$$\boldsymbol{v}_{ia}(t) = U \begin{pmatrix} x_{ia}(t) \\ s_{ia}(t-1) \end{pmatrix} + \boldsymbol{b}. \tag{7}$$

The branch input, $\boldsymbol{v}_{ia}(t)$, combines the gated feature, $x_{ia}(t)$, and the previous emitted spike, $s_{ia}(t-1)$. The previous spike provides top-down feedback from the spiking output to the dendritic branches. The projection weight, $U \in \mathbb{R}^{4\times 2}$, and the bias, $\boldsymbol{b} \in \mathbb{R}^4$, are trainable.

The provisional branch state, $\widehat{\boldsymbol{h}}_{ia}(t)$, balances the previous state and the current branch input through the retention vector, $\boldsymbol{\beta}_i$:

$$\begin{aligned} \widehat{\boldsymbol{h}}_{ia}(t) = \boldsymbol{\beta}_i \odot \boldsymbol{h}_{ia}(t-1) \\ + (1 - \boldsymbol{\beta}_i) \odot \boldsymbol{v}_{ia}(t). \end{aligned} \tag{8}$$

$$\begin{aligned} \boldsymbol{h}_{ia}(t) &= \boldsymbol{h}_{ia}(t-1) + \big(1 - \eta_a(1 - g_i(t))\big) \\ &\quad \cdot \big(\widehat{\boldsymbol{h}}_{ia}(t) - \boldsymbol{h}_{ia}(t-1)\big), \\ H_{ia}(t) &= \sum_{b=1}^{4} h_{iab}(t). \end{aligned} \tag{9}$$

The branch state, $\boldsymbol{h}_{ia}(t)$, moves toward the provisional branch state, $\widehat{\boldsymbol{h}}_{ia}(t)$, according to the gate and the trainable modulation weight, η_a. The branch index, b, selects an entry of $\boldsymbol{h}_{ia}(t)$; the membrane input, $H_{ia}(t)$, sums all four entries.

Membrane layer. The provisional membrane voltage, $\widehat{V}_{ia}(t)$, balances the previous voltage and the current membrane input through the retention coefficient, λ_i, then subtracts a reset proportional to the previous spike:

$$\begin{aligned} \widehat{V}_{ia}(t) = \lambda_i V_{ia}(t-1) + (1 - \lambda_i) H_{ia}(t) \\ - \vartheta_a s_{ia}(t-1). \end{aligned} \tag{10}$$

$$\begin{aligned} V_{ia}(t) = V_{ia}(t-1) + \big(1 - \xi_a(1 - g_i(t))\big) \\ \cdot \big(\widehat{V}_{ia}(t) - V_{ia}(t-1)\big). \end{aligned} \tag{11}$$

$$e_{ia}(t) = \mathrm{ind}(V_{ia}(t) > \vartheta_a),$$
$$s_{ia}(t) = g_i(t) e_{ia}(t). \tag{12}$$

The membrane voltage, $V_{ia}(t)$, moves toward the provisional voltage, $\widehat{V}_{ia}(t)$, according to the gate and the trainable modulation weight, ξ_a. The firing threshold, ϑ_a, is derived from a trainable parameter. The indicator function, ind, produces the binary firing event, $e_{ia}(t)$, which the gate scales to produce the emitted spike, $s_{ia}(t)$.

2.3 Spike Grouping and Mask Generation

We mean-center and L2-normalize the final 512 emitted spike samples for each oscillator component. The component correlation, ρ_{ija}, sums the products of corresponding samples in the normalized traces:

$$\rho_{ija} = \mathrm{clip}\left(\sum_{m=1}^{512} r_{ia,m}\, r_{ja,m}, -1, 1\right), \tag{13}$$

where the sample index, m, runs over the retained time window. To obtain the patch affinity, Q_{ij}, we replace negative correlations with zero and multiply the resulting values across the four components. We form connected components by linking patch pairs with $Q_{ij} \geq 0.5$, discard components smaller than 32 patches, and treat the largest remaining component as background.

We start the retained foreground patches as singleton groups and merge spatially adjacent groups using

$$E_{ij} = \left(1 + \frac{N \bar{A}_{ij}}{G}\right) \log \frac{Q'_{ij}}{1 - Q'_{ij}},$$
$$\Delta(\mathcal{C}, \mathcal{D}) = \sum_{i \in \mathcal{C}} \sum_{j \in \mathcal{D}} E_{ij}. \tag{14}$$

The stabilized affinity, Q'_{ij}, determines the sign of the merge evidence, E_{ij}. The merge score, $\Delta(\mathcal{C}, \mathcal{D})$, sums the evidence between the candidate patch groups, $\mathcal{C}$ and $\mathcal{D}$. Groups are merged while their score is positive. Each final group produces an object mask.

3 Experiments

3.1 Protocol

All models are evaluated by one function on one representation. Ground truth is quantised to a 64×64 patch grid and scored there; a prediction is *never* upsampled to pixels to be compared. A prediction coarser than the grid is lifted by an exact `repeat`, and one finer than the grid is reduced by patch majority vote, so every method is scored on the same object. The ceiling is therefore 1.0 by construction, and these numbers are not directly comparable with pixel-protocol figures reported elsewhere.

We report three metrics and never combine them into a single score: foreground adjusted Rand index (FG-ARI), foreground IoU (FG-IOU) and matched-object IoU (OBJ-IOU). Background is label 0 on both sides, and the foreground-only metrics exclude patches the ground truth calls background.

Matched exposure. Every model is trained on the *same* $10{,}000$ CLEVR images for 10 epochs, i.e. $100{,}032$ exposures. Where a baseline's published recipe differs, only its epoch count is changed; architecture and optimiser are left at the authors' values. Tetrominoes and Multi-dSprites are never trained on: for every row in Table 1 those two columns are zero-shot transfer from the CLEVR-trained model.

Readouts that require a cluster count. Several baselines produce a continuous embedding and obtain masks by clustering it. We found that the reference implementations of CAE, CAE++, CtCAE and SynCx set the number of clusters from the *per-image ground-truth object count*,

$$k_i \;=\; \big|\mathrm{unique}(y_i)\big|,$$

which makes their published figures oracle-assisted. For the comparison to be meaningful we removed this: k is a per-dataset constant selected on a validation split, never read from the evaluated image. Cluster identities are arbitrary, so the mapping of one cluster to background is likewise fixed by a rule chosen on validation. No hyperparameter, threshold or rule is selected on the reported split.

Minimum region size. Our readout emits a sizeable number of single-patch regions: 22.8% of all predicted regions on CLEVR validation, 27.0% on Tetrominoes and 56.2% on Multi-dSprites, where the *median* predicted region is one patch. They arise after the synchrony grouping, when the greedy merge splits a spatially scattered synchronous component into pieces, and they inflate the object count without materially affecting the partition. We therefore discard any predicted region smaller than a fraction f of that image's predicted foreground, reassigning it to background. The rule is scale-free, reads no ground truth, and f is a single constant fixed on CLEVR validation as the largest value whose FG-ARI loss stays under 0.005 and whose matched-object IoU loss stays under 0.01, giving $f = 0.01$. The same f is applied unchanged to Tetrominoes and Multi-dSprites — no target-domain data is used to set it — and to every baseline.

3.2 Generalisation across datasets

Table 1 is the central result. Every model sees only CLEVR during training; the Tetrominoes and Multi-dSprites columns measure transfer.

Our readout is ahead in all nine cells. The comparison is deliberately pessimistic: the *best baseline per cell* is a composite of three different architectures, and no single competitor achieves that row. It is also notable that *which* baseline comes second depends on the dataset — AKOrN (conv) on Tetrominoes, AKOrN (attn)+up-tiling on Multi-dSprites, ItrSA on CLEVR matched-object IoU — whereas our model is first everywhere.

Where the margin is thin. The smallest margin in the table is Tetrominoes foreground IoU, $+0.0410$. Every other margin lies between 0.128 and 0.326. We flag this cell explicitly rather than quoting only the aggregate.

Predicted foreground. A single diagnostic separates grouping from tiling. Against a true foreground fraction of

Model	CLEVR (in-domain)			Tetrominoes (zero-shot)			Multi-dSprites (zero-shot)		
	ARI	F-IOU	O-IOU	ARI	F-IOU	O-IOU	ARI	F-IOU	O-IOU
Ours	**.8408**	**.7528**	**.6387**	**.8568**	**.7950**	**.8170**	**.6966**	**.8190**	**.7625**
AKOrN (attn) + up-tiling	.6326	.3468	.3636	.5484	.7230	.4167	.5505	.6539	.6348
AKOrN (conv)	.6046	.4219	.2769	.7202	.7540	.5913	.4055	.6834	.5434
AKOrN (attn)	.6027	.3327	.3750	.5676	.7383	.4687	.4343	.6019	.5751
ItrSA	.5794	.2787	.3853	.6319	.7137	.5687	.4817	.2489	.6093
CAE++	.2838	.2193	.0882	.2715	.2452	.1901	.2648	.1048	.1125
Slot Attention	.2872	.2220	.0801	.1739	.2192	.1071	.1930	.0876	.0680
CtCAE	.2357	.2148	.0964	.2681	.2420	.2034	.3079	.1158	.1609
CAE	.1091	.2008	.0634	.1526	.2305	.1192	.1200	.1038	.0630
Best baseline, per cell	.6326	.4267	.3853	.7202	.7540	.5913	.5505	.6834	.6348
Our margin	**+.2082**	**+.3261**	**+.2534**	**+.1366**	**+.0410**	**+.2257**	**+.1461**	**+.1356**	**+.1277**

Table 1: All models trained on the same $10{,}000$ CLEVR images for 10 epochs and evaluated on 320 held-out images per dataset by the same function. Tetrominoes and Multi-dSprites are zero-shot. ARI / F-IOU / O-IOU are foreground ARI, foreground IoU and matched-object IoU on the 64×64 patch grid. Every model, ours included, has the minimum-region-size rule of Section 3.1 applied at the single value $f = 0.01$ fixed on CLEVR validation. It leaves the baseline rows unchanged to three decimals except AKOrN (conv) on CLEVR foreground IoU, which improves from $.4219$ to $.4267$; the penultimate row uses the better of the two for every baseline cell we measured. That row takes the best baseline *in each cell separately*, which no single baseline attains; our margin is measured against that composite. Single training seed throughout.

0.2115 on CLEVR and 0.0943 on Multi-dSprites, our readout predicts 0.1988 and 0.0827 — the only method that does not over-claim — while baselines predict between 0.41 and 0.86 on CLEVR. A model that labels most of the image as object can score well on a foreground-only partition measure while failing the IoU metrics, which is what the F-IOU column shows.

3.3 Are the baselines handicapped?

Two objections to Table 1 deserve measurements rather than argument: that the baselines are evaluated away from their published setting, and that the matched budget starves them.

Released weights. Only Slot Attention ships trained weights; the AKOrN, CAE and SynCx repositories release data but no checkpoints. Evaluated directly, at roughly $320\times$ our exposure budget, its released model scores $.3747/.2219/.1165$ on CLEVR, $.4742/.2381/.2355$ on Tetrominoes and $.3205/.0980/.1204$ on Multi-dSprites. We remain ahead in all nine cells, by 0.377 to 0.720, so the comparison does not rest on our retraining of it. The extra data also helps it far more *off* its training distribution than on it — worth $+.0875$ FG-ARI on CLEVR but $+.3003$ on Tetrominoes and $+.1275$ on Multi-dSprites — so a data-efficiency conclusion drawn from an in-domain column alone would not transfer.

Budget. Comparing each baseline against its own published number, by its own evaluation code, isolates the effect of the shortened schedule. Two of the six are *not* budget-limited: AKOrN (conv) reaches 60.5 at $1/210$ of its schedule against a published 59.0 ± 4.3, and CAE reaches 10.3 at $1/64$ against a published 4.0. For those two the margin in Table 1 is not an artefact of a short schedule. The other four are limited — AKOrN (attn) 60.4 against 91.0 ± 0.5, ItrSA 58.1 against 82.5 ± 1.7, CAE++ 16.7 against 27.0, and CtCAE 12.0 against 54.0 — and against those the result should be read as a data-efficiency claim. We state both readings rather than only the favourable one. Full figures are in the supplementary document.

T	FG-ARI	FG-IOU	OBJ-IOU	Δ FG-ARI
1024	.8421	.7545	.6410	—
512	.8402	**.7594**	.6340	$-.0019$
256	.8159	.7358	.6084	$-.0262$
128	.5796	.4988	.3985	$-.2625$

Table 2: CLEVR, model fixed, readout window $T/2$. Halving the rollout is free; a quarter costs 0.02–0.03; an eighth fails. This table and the scaling figures below are reported *without* the minimum-region-size rule, because both are within-model comparisons at a fixed readout and the rule would shift every row by the same small amount.

3.4 Rollout length

The readout integrates spiking activity over T simulation steps and reads the second half of the window. Table 2 varies T with the model fixed.

Halving the rollout costs 0.0019 FG-ARI — below any difference we can resolve at one seed — and *improves* foreground IoU. Performance degrades gently to $T = 256$ and collapses at $T = 128$, where the model also under-segments, predicting a foreground fraction of 0.1148 against a truth of 0.2115. We therefore use $T = 512$ as the default operating point and $T = 1024$ for the reported numbers. Shorter windows also fragment more: mean predicted regions per image rise from 8.0 at $T = 1024$ to 13.3 at $T = 256$, against 6.36 true objects.

Scaling with training data. Continuing training on additional unseen CLEVR images, in steps of 1,000, separates the three metrics. FG-ARI rises from .8421 at 10,000 images to .8469 at 14,000 and then falls to .8459 at 15,000; its step-to-step changes are $+.0005$, $+.0023$, $+.0011$, $+.0009$, $-.0010$. Foreground IoU and matched-object IoU do not turn over the same range, gaining $+.0157$ and $+.0124$ and still rising at 15,000. More in-domain data therefore stops improving *how* patches are grouped well before it stops improving *where the boundaries fall*. Mean predicted regions hold at 7.7–8.0 throughout, against 6.36 true objects, so the extra data sharpens the partition without changing how much it splits. A single 0.0010 decrease at one seed is not proof of saturation, and we report it as the shape of the curve rather than a converged value.

In-domain data is not the most useful data. The same 5,000 additional images buy more than twice as much when they come from a different dataset. Continuing on CLEVR raises CLEVR FG-ARI by $+.0038$; continuing on Tetrominoes instead raises it by $+.0084$, a $2.2\times$ larger gain at an identical budget. Together with the transfer result in Table 1, this argues that the model is not fitting a domain but learning a grouping rule that the extra renders refine.

3.5 Limitations

Object count. Our readout groups patches better than it counts objects. With the minimum-region-size rule it predicts 5.15 objects per image on CLEVR against a true 6.36, matching the exact count on 25.0% of images; 2.64 against 2.49 on Multi-dSprites (53.1%); and 5.28 against exactly 3 on Tetrominoes (9.4%). Without the rule the same model predicts 8.04, 6.62 and 21.07, matching on 17.5%, 28.1% and *none* of the 320 Tetrominoes images, so most of the count error was single-patch fragments rather than genuine oversegmentation. Letting f be chosen on each dataset's own validation split instead of transferred from CLEVR removes most of the remainder — $f = 0.10$ gives 3.02 predicted objects on Tetrominoes, 86.9% exact, and raises foreground IoU further to .8066 — but that uses target-domain labels, so we report the transferred value as the headline.

Two cautions about this metric. First, Tetrominoes contains exactly three objects in every image, so a model that always answers three is exactly right on every image without grouping anything: the fixed-k baselines all score 100% there, CtCAE included at FG-ARI 0.0756. The count is only informative where the true count varies, and on Multi-dSprites, where it does, the same baselines score 0% exact against our 53.1%. Second, the residual error is a predicted consequence of the objective: the model is trained without a reconstruction term, and a reconstruction loss is what penalises splitting one object into two parts that must then be reconstructed independently. Every baseline in Table 1 optimises such a term; ours does not, which is also what makes it cheap to train.

Seeds. Every number reported here is a single training seed. We measured the effect of reseeding one baseline, which moved FG-ARI by up to 0.134 on CLEVR and 0.009 on Multi-dSprites. All margins in Table 1 exceed that range, but this is evidence about baseline variance and not an error bar on our own model.

Protocol. The patch protocol is not comparable with pixel-protocol numbers reported in the cited works, and the held-out split is reserved from the training corpus rather than an independent benchmark test set. Figures from other papers appear in this section only as each model's comparison with itself (Section 3.3), never as entries in Table 1.

References

Dan, T.; and Wu, G. 2026. From Cortical Synchronous Rhythm to Brain Inspired Learning Mechanism: An Oscillatory Spiking Neural Network with Time-Delayed Coordination. *arXiv preprint arXiv:2605.01656*.